EF Core – ASP.NET Essentials II

1. Overview

In this exercise, students will focus on implementing the **backend logic** of the CinemaApp project. The primary objective is to build **CRUD (Create, Read, Update, Delete) operations** for managing data while **working directly with DbContext via Dependency Injection (DI).** This will provide a hands-on understanding of handling database interactions efficiently in an ASP.NET Core MVC application.

Learning Objectives

By completing this exercise, students will:

- Implement **CRUD functionality** in controllers for key entities
- Work with **DbContext** using **Dependency Injection**
- Understand how to **structure controller logic** for **interacting with the database**
- Learn how to **manage data import and export** using helper methods

Best Practices and Approach

A common **best practice in ASP.NET Core applications** is to use:

- **Repository Pattern**
- **Service Layer**

to **separate concerns** and **avoid direct interaction with DbContext in controllers**.

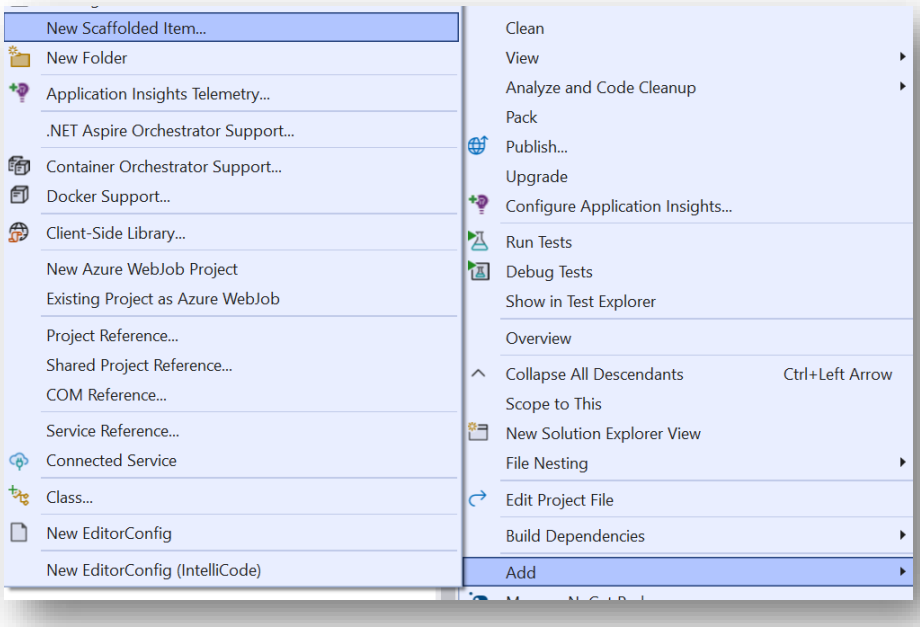
However, **to keep the application simple** and **avoid introducing too much information at the beginning**, we will directly interact with DbContext in this exercise.

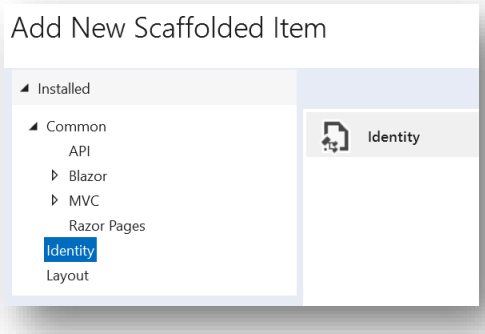
Later, students can **refactor** the application to follow best practices.

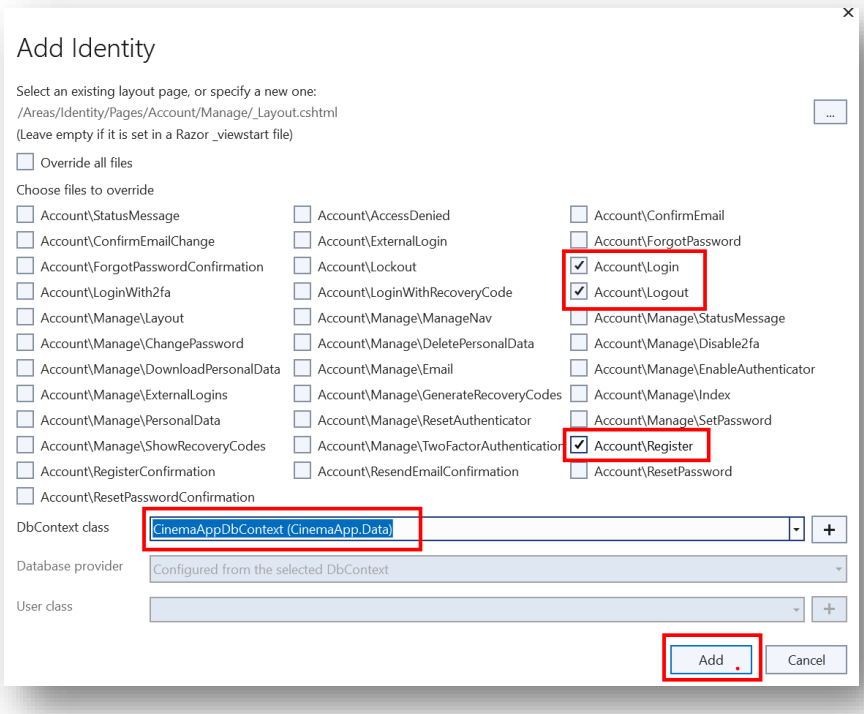
2. Identity Setup

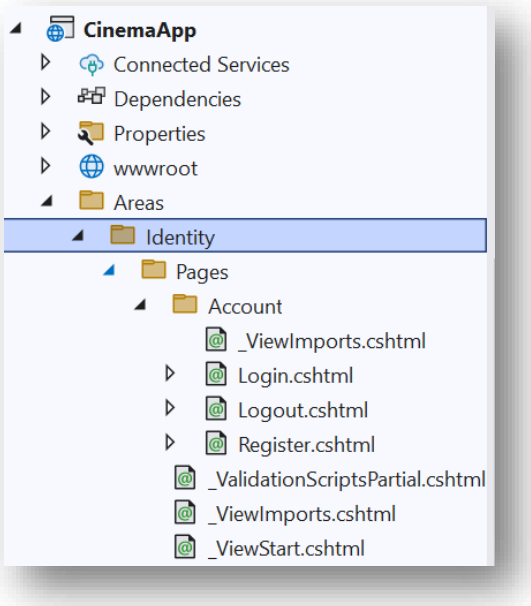
Scaffolding Identity

- Right-click on the **Project** in **Solution Explorer**
- Select **Add > New Scaffolded Item**
- Choose **Identity**
- Select **Account: Login, Register, Logout**
- Choose **`CinemaAppDbContext`** as the data context
- Click **Add**









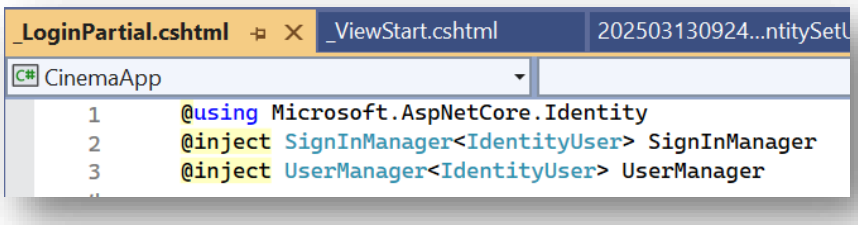
Change IdentityUser to ApplicationUser

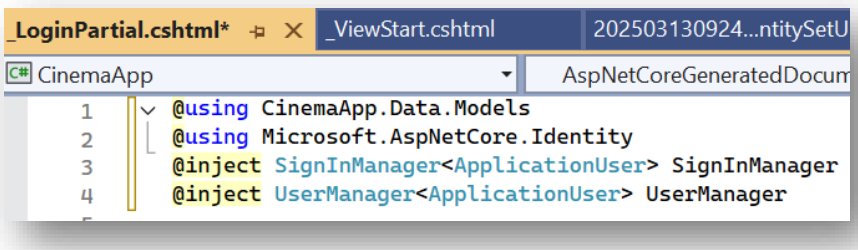
After scaffolding, the Identity pages (such as **`Register.cshtml`** and **`Login.cshtml`**) will **use IdentityUser by default**. However, if you have a custom **`ApplicationUser`** class, you must **update all** occurrences of IdentityUser in the scaffolded files.

- Update **`Register.cshtml.cs`, `Login.cshtml.cs`** and **`Logout.cshtml.cs`**
- **Apply Migrations**

Register a User and Extract User IDs from SSMS

- **Editing `_LoginPartial.cshtml`**

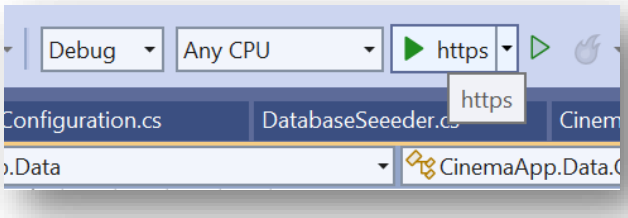


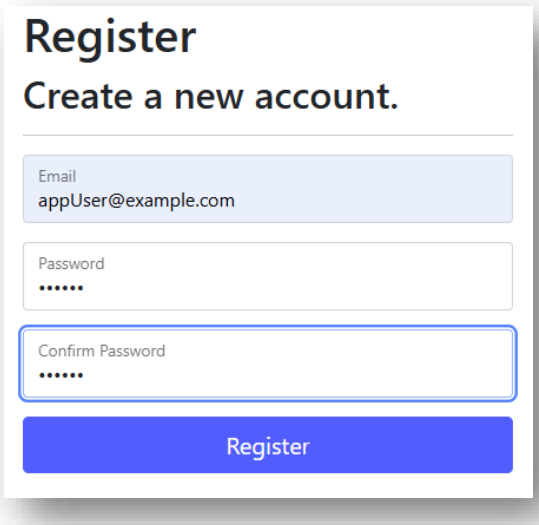


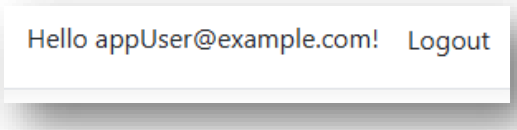
  By default, the **`_LoginPartial.cshtml`** file injects **`UserManager<IdentityUser>`** and **`SignInManager<IdentityUser>`**.
  However, since we are using **`ApplicationUser`**, you must **update this file** to prevent runtime errors.
- **Remove Password Restrictions**

```
builder.Services.AddIdentity<ApplicationUser, IdentityRole<Guid>>(options =>
{
    options.SignIn.RequireConfirmedAccount = true;
    options.Password.RequireDigit = false;
    options.Password.RequiredLength = 1;
    options.Password.RequireLowercase = false;
    options.Password.RequireUppercase = false;
    options.Password.RequireNonAlphanumeric = false;
})
    .AddEntityFrameworkStores<CinemaAppDbContext>()
    .AddDefaultTokenProviders();
```

- **Run the Application** and navigate to the **Register page:**







	Id	UserName
1	751CF4BD-288F-48DD-ADAE-37FD5A70BB10	admin@example.com
2	F964C689-0C3E-42FC-8155-915538C0083F	appManager@example.com
3	680F3068-C4F0-46EF-BE81-C95BF21A992F	appUser@example.com

3. Creating a DataProcessor static class

Seeding Files for Import

- Additional **`movies`** to import:

- **`CinemasMovies`** relations to import:

- Tickets to import:

Contains a **list of predefined movies** that will be imported into the database

The **`tickets.xml`** file includes ticket data with **placeholders for `UserId` elements**.

These values are initially **empty** and can be updated once **users are registered** through the default **`**ASP.NET Identity**`** registration functionality.

This ensures that ticket ownership can be correctly assigned.

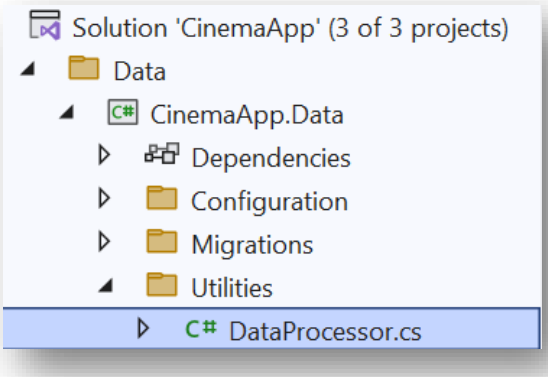
Assigning Users to Tickets in the XML File

Now that we have users created in the database, we need to **update the tickets.xml** file by **assigning Id values** to each ticket.

```xml
<?xml version="1.0" encoding="UTF-8"?>
<Tickets>
    <Ticket>
        <Id></Id>
        <Price>12.50</Price>
        <CinemaId></CinemaId>
        <MovieId></MovieId>
        <UserId></UserId>
    </Ticket>
    <Ticket>
        <Id></Id>
        <Price>15.00</Price>
        <CinemaId></CinemaId>
        <MovieId></MovieId>
        <UserId></UserId>
    </Ticket>
    <Ticket>
        <Id></Id>
        <Price>10.00</Price>
        <CinemaId></CinemaId>
        <MovieId></MovieId>
        <UserId></UserId>
    </Ticket>
</Tickets>
```

Where to Place the DataProcessor Class?

The DataProcessor class should be placed in a logical and reusable location within your ASP.NET Core project

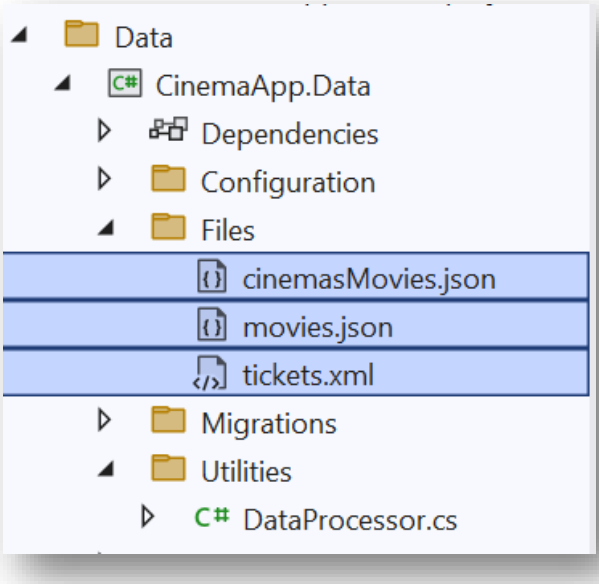


Creating DataProcessor Class Empty Methods

```
public static class DataProcessor
{
    1 reference
    public static void ImportMoviesFromJson(CinemaAppDbContext dbContext)
    {
        throw new NotImplementedException();
    }

    1 reference
    public static void ImportCinemasMoviesFromJson(CinemaAppDbContext dbContext)
    {
        throw new NotImplementedException();
    }

    1 reference
    public static void ImportTicketsFromXml(CinemaAppDbContext dbContext)
    {
        throw new NotImplementedException();
    }
```

```
private static string GetProjectDirectory()
{
    string assemblyFolder = AppContext.BaseDirectory;

    string solutionFolder = Path.GetFullPath(Path.Combine(assemblyFolder, @"../../../../"));

    string fullPath = Path.Combine(solutionFolder, "CinemaApp.Data", "Files");
    return fullPath;
}
```

Create a Folder for Data Files



4. Triggering DataProcessor When the App Starts

- **To execute the DataProcessor at startup, we need to modify Program.cs to call DataSeeder.**
- **This will ensure that the data is imported when the application starts.**

```
using (var scope = app.Services.CreateScope())
{
    var services = scope.ServiceProvider;
    var dbContext = services.GetRequiredService<CinemaAppDbContext>();

    DataProcessor.ImportMoviesFromJson(dbContext);
    DataProcessor.ImportCinemasMoviesFromJson(dbContext);
    DataProcessor.ImportTicketsFromXml(dbContext);
}
app.Run();
```

Comment Out the Following Lines

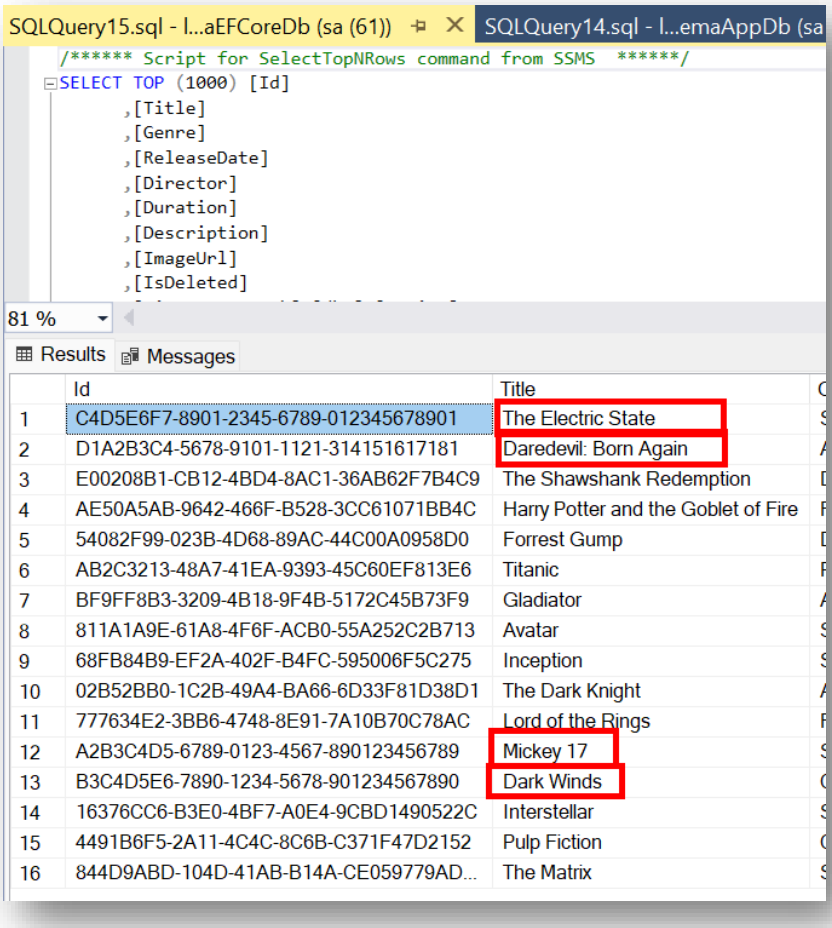
```
using (var scope = app.Services.CreateScope())
{
    var services = scope.ServiceProvider;
    var dbContext = services.GetRequiredService<CinemaAppDbContext>();

    //DataProcessor.ImportMoviesFromJson(dbContext);
    //DataProcessor.ImportCinemasMoviesFromJson(dbContext);
    //DataProcessor.ImportTicketsFromXml(dbContext);
}
app.Run();
```

- We will **uncomment one by one** the **`DataProcessor`** methods, and seed the new entries in the database

Import Movies From Json

```
public static void ImportMoviesFromJson(CinemaAppDbContext dbContext)
{
    string filePath = Path.Combine(GetProjectDirectory(), "movies.json");

    string json = File.ReadAllText(filePath);
    var movies = JsonSerializer.Deserialize<List<Movie>>(json, new
     JsonSerializerOptions
    {
        PropertyNameCaseInsensitive = true
    });

    dbContext.Movies.AddRange(movies);
    dbContext.SaveChanges();
}
```

Ensure Movies are Imported Successfully

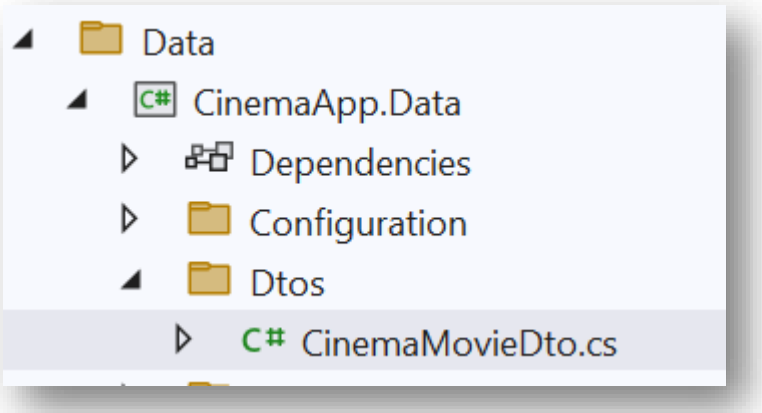


	Id	Title
1	C4D5E6F7-8901-2345-6789-012345678901	The Electric State
2	D1A2B3C4-5678-9101-1121-314151617181	Daredevil: Born Again
3	E00208B1-CB12-4BD4-8AC1-36AB62F7B4C9	The Shawshank Redemption
4	AE50A5AB-9642-466F-B528-3CC61071BB4C	Harry Potter and the Goblet of Fire
5	54082F99-023B-4D68-89AC-44C00A0958D0	Forrest Gump
6	AB2C3213-48A7-41EA-9393-45C60EF813E6	Titanic
7	BF9FF8B3-3209-4B18-9F4B-5172C45B73F9	Gladiator
8	811A1A9E-61A8-4F6F-ACB0-55A252C2B713	Avatar
9	68FB84B9-EF2A-402F-B4FC-595006F5C275	Inception
10	02B52BB0-1C2B-49A4-BA66-6D33F81D38D1	The Dark Knight
11	777634E2-3BB6-4748-8E91-7A10B70C78AC	Lord of the Rings
12	A2B3C4D5-6789-0123-4567-890123456789	Mickey 17
13	B3C4D5E6-7890-1234-5678-901234567890	Dark Winds
14	16376CC6-B3E0-4BF7-A0E4-9CBD1490522C	Interstellar
15	4491B6F5-2A11-4C4C-8C6B-C371F47D2152	Pulp Fiction
16	844D9ABD-104D-41AB-B14A-CE059779AD...	The Matrix

5. Next Steps for Import:

ImportCinemasMoviesFromJson

- Create a **`CinemaMovieDTO`**




```
public class CinemaMovieDto
{
    0 references
    public string Movie { get; set; } = null!;
    0 references
    public string Cinema { get; set; } = null!;
    0 references
    public int AvailableTickets { get; set; }
    0 references
    public bool IsDeleted { get; set; }
    0 references
    public string Showtimes { get; set; } = null!;
}
```

```
using (var scope = app.Services.CreateScope())
{
    var services = scope.ServiceProvider;
    var dbContext = services.GetRequiredService<CinemaAppDbContext>();

    //DataProcessor.ImportMoviesFromJson(dbContext);
    DataProcessor.ImportCinemasMoviesFromJson(dbContext);
    //DataProcessor.ImportTicketsFromXml(dbContext);
}
app.Run();
```

ImportTicketsFromXML